

Comprendre quand chaque modèle de facturation est avantageux

Deux modèles fondamentalement différents

Subscription : vous payez un montant mensuel fixe et consommez dans la limite d'un budget de tokens. Le coût est prévisible, mais le budget est plafonné et les limites sont délibérément opaques.

API : facturation au token, sans plafond. Le coût est proportionnel à l'usage. Idéal pour un usage irrégulier ou très élevé, mais sans protection contre les dépassements.

La distinction pratique : la subscription favorise les développeurs solo avec un usage quotidien régulier, l'API favorise les équipes, les pipelines CI/CD et les usages à fort volume.

Comment fonctionnent les limites subscription

Les subscriptions utilisent un modèle hybride : une fenêtre glissante de 5 heures et un cap hebdomadaire. Les deux s'appliquent simultanément. La facturation est annoncée en « messages » mais correspond en réalité à des tokens, ce qui rend les limites difficiles à anticiper.

Point critique : le ratio Opus/Sonnet. Opus consomme 8 à 10 fois plus de quota que Sonnet pour un travail équivalent. Un utilisateur Max 20x qui utilise Opus intensivement peut épuiser son quota hebdomadaire en 24 à 40 heures de travail réel, malgré un abonnement premium.

Stratégies par plan

Plan	Recommandation
Pro	Sonnet uniquement, sessions par batch, éviter le context bloat
Max 5x	Sonnet par défaut, Opus pour l'architecture et le debug complexe
Max 20x	Plus de liberté Opus, mais surveiller la consommation hebdomadaire
API	Haiku pour les tâches mécaniques, Sonnet pour le dev, Opus pour l'audit

Le pattern OpusPlan pour économiser

Sur un budget limité, le pattern le plus efficace consiste à utiliser Opus pour la planification (où la qualité du raisonnement compte), puis Sonnet ou Haiku pour l'exécution mécanique.

```
# Phase 1 : planification (Opus)  claude  --model opus
"Analyse l'architecture et propose un plan de migration"
# Phase 2 : exécution (Sonnet/Haiku)  claude  --model sonnet
"Implémente le plan selon les specs définies"
```

Ce pattern est automatisé via `/model opusplan`.

Monitoring de consommation

```
/cost
# Coût et contexte de la session en cours  /status
# Modèle + contexte + coût résumé
# Outil communautaire pour l'historique cross-sessions
ccusage  # Vue d'ensemble toutes périodes
ccusage  --today  # Coût du jour  ccusage
--month  # Coût du mois  ccusage
--model-breakdown  # Par modèle
```

Anthropic ne fournit pas de métriques temps réel dans l'interface. `ccusage` comble ce manque.

Quand l'API gagne

La subscription atteint ses limites dans trois cas : les équipes où plusieurs développeurs consomment simultanément, les pipelines CI/CD avec un volume de PRs élevé, et les projets avec des sessions longues quotidiennes qui épuisent le budget avant la fin de la semaine. Dans ces contextes, la facturation API (avec Haiku pour les tâches mécaniques) revient souvent moins cher que prévu, surtout avec une stratégie d'escalade progressive.

Coûts typiques en API (ordre de grandeur)

Session type	Sonnet	Opus
Bug fix / PR review	~\$0.23	~\$0.38
Refactoring d'un module	~\$0.75	~\$1.25
Review CI/CD par PR (Haiku)	~\$0.02	N/A